

WEB TECHNOLOGIES

CSE-4004



LECTURE 3: Arrays & Functions



By: Dr.MD.Sirajuddin

Arrays

- An array is a data structure that stores a collection of elements of the same or different data types.
- Store many values/elements under a single name
- Each element is accessed using an index.
- Arrays can grow or shrink dynamically.

Declaring an Array

1. *let a=[10,20,30], b=[10, "hai",2.5];*
2. *let a=new Array(10,20,30);*
3. *let a=new Array(5); // array of length 5*
4. *const a=["CSE", 10];*

Accessing Elements

```
let course = ["AI", "ML", "CS"];  
document.write("<h1>" + course[0] + "</h1>");
```

Changing an Array Element

```
course [0] = "NS", ;  
document.write("<h1>" + course[0] + "</h1>");
```

Arrays as an Object: Arrays are special type of objects

```
Mixed_array= [Date.now, my_func, course];
```

An array can hold objects, reference to functions & arrays.

my_func //reference to function

my_func() // execute function

Arrays as an Object: Arrays are special type of objects

```
Mixed_array=[Date.now(),my_funct, course];  
[1768481192470, f, Array(3)]  
Mixed_array[1]( ); //executes function
```

Array with Looping Statement

```
let len=course.length;  
for(let i=0; i<len;i++)  
{  
    document.write(course[i]+"<br>");  
}
```

2D-ARRAY

- *let mat= [[1,2,3], [4,5,6], [7,8,9]];*
document.write(mat[0][1]);
- **Using For Loop**
for (let i = 0; i < mat.length; i++)
{
for (let j = 0; j < mat[i].length; j++)
{
document.write(mat[i][j]);
}
}

Multidimensional Array

- A multidimensional array is an array that contains more than one dimension.
- Its elements are accessed using two or more indices.

➤ **let arr = [[[1, 2], [3, 4]], [[5, 6], [7, 8]]];**

Arr[1][0][1] // 6

for (let i = 0; i < arr.length; i++)

{

for (let j = 0; j < arr[i].length; j++)

{

for (let k = 0; k < arr[i][j].length; k++)

{

console.log(arr[i][j][k]);

} } }

ARRAY FUNCTIONS

Method	Syntax	Example	Output
length	array.length	Let a=[1,2]; a.length	2
push()	array.push(element)	let a=[1,2]; a.push(3);	[1,2,3]
pop()	array.pop()	let a=[1,2,3]; a.pop();	returns 3
shift()	array.shift() <i>removes first element</i>	let a=[1,2,3]; a.shift();	returns 1
unshift()	array.unshift(element) <i>Adds element at start</i>	let a=[2,3]; a.unshift(1);	[1,2,3]
concat()	arr1.concat(arr2)	let a=[1,2]; let b=[3,4]; let c=a.concat(b);	returns c=[1,2,3,4]
slice()	array.slice(start, end)	let a=[1,2,3,4]; b=a.slice(1,3)	returns shallow copy [2,3]
splice()	arr.splice(start, deleteCount, items...)	let a=[1,2,3]; a.splice(1,1,9);	a=[1,9,3]
indexOf()	array.indexOf(value)	course.indexOf("ML")	1

ARRAY FUNCTIONS

Method	Syntax	Example	Output
lastIndexOf()	array.lastIndexOf()	let a=[1,2,2]; a.lastIndexOf(2);	Returns 2
includes()	array.includes(value)	let a=[1,2,3]; a.includes(2);	true
sort()	arr.sort()	let a=[3,1,2]; a.sort();	[1,2,3]
reverse()	arr.reverse()	let a=[1,2,3]; a.reverse();	[3,2,1]
join()	arr.join(separator)	let a=[1,2,3]; a.join("-");	"1-2-3" returns a string

ARRAY FUNCTIONS

- The **flat()** method creates a new array with all sub-array elements concatenated into it.
- Converting nested array to single array.
- By default, it flattens the array to a depth of 1.

Example 1

```
let cube = [ [ [1, 2], [3, 4] ], [ [5, 6], [7, 8] ] ];
```

```
cube.flat() //default level 1
```

```
Output: [ [1, 2], [3, 4], [5, 6], [7, 8] ]
```

Example 2

```
let cube = [ [ [1, 2], [3, 4] ], [ [5, 6], [7, 8] ] ];
```

```
cube.flat(2) //default level 1
```

```
Output: [1,2,3,4,5,6,7,8]
```

ARRAY FUNCTIONS

- The **filter()** method creates a new array with elements that satisfies the given condition

Example 1

```
let a=[1,2,3];  
let b=a.filter(n => n>1);
```

Output: [2,3]

- The **find()** method returns a first element that satisfies the given condition

Example 1

```
let a=[1,2,3];  
a.find(n => n>1);
```

Output: 2

ARRAY FUNCTIONS

- The **findIndex()** method returns index of first element that satisfies the given condition

Example 1

```
let a=[1,2,3];  
a.findIndex(n => n>1);
```

Output: 1

- The **every()** checks if all elements satisfies a given condition.

Example 1

```
let a=[2,4,6];  
a.every(n => n%2===0);
```

Output: true

ARRAY FUNCTIONS

- The **some()** checks if any element satisfies a given condition.

Example 1

```
let a=[1,2,3];  
a.some(n => n%2===0);
```

Output: true

ARRAY FUNCTIONS

- The **map()** creates an array by applying a function to every element in original array.
- It returns same length array without changing original array.

Example 1

```
let a=[1,2,3];  
b=a.map(n => n*3);  
console.log(b)
```

Output: [3,6,9]

ARRAY FUNCTIONS

- The **reduce()** reduces an array to single value by applying a function on each element of an array.
- It uses an accumulator.

Syntax: `array.reduce((acc, cur) => acc + cur);`

`acc` points to first element in array

`cur` points to second element in array

Example 1

```
let a=[1,2,3];
```

```
b=a.reduce((acc, cur) => acc + cur); // acc=1, cur=2 (element)
```

```
console.log(b)
```

Output: 6

Example 2

```
b=a.reduce((acc, cur) => acc + cur,0); // acc=0, cur=1 (ele)
```

Output: 6